

Group Forecast and Furious
Casey Barrasso, Joshua Previte, Giovanni Jones

This week, we identified three separate techniques to improve data quality.

Firstly we created lagged versions of the raw AAPL share price, which means it captures the actual numerical value of the stock at previous time steps. Specifically, for each lag from 1 to 5, the `.shift(i)` function moves the price data down by `i` rows, so that each new column represents the stock price from prior days. For example, `lag1` contains yesterday's price, `lag2` contains the price from two days ago, and so on. This approach produces continuous-valued features, allowing a model to observe not just whether the price went up or down, but also by how much. As a result, the model can learn patterns related to magnitude, volatility, and trends over time. These features are particularly useful for more complex models that benefit from richer numerical input, as they preserve the full information contained in the original price series. In contrast, the original data that we fed into the model used lagged binary values which involved first transforming the price into a directional signal, typically represented as 1 (price increased) or 0 (price decreased or stayed the same), and then applying the same lagging process. This results in features that only capture the direction of past movements rather than their size.

The second step is a technique called SMOTE, which creates synthetic observations for the minority class. This is particularly important for this dataset because the "increase" stock price observations outweighed the "decrease" observations by 53% to 47%. This allowed the model to create a baseline accuracy of 53% simply by predicting all observations as "increase". SMOTE evens out the discrepancy between the classes, and reduces the possibility of a model optimizing accuracy by predicting a single class.

We also used the bootstrapping method to see if that would have a positive impact on the model's performance. Bootstrapping is a resampling technique that creates new training datasets by randomly sampling the original dataset with replacement. This means that the same data points can appear multiple times within a sample. Some data points may also not be included at all. Bootstrapping is often used when a dataset is small and more observations would be helpful for modeling. In our case, we are working with daily stock price data for a single company. Since there is a finite number of data points for our case (stock data only exists for as long as a company has been public) bootstrapping seemed like a logical way to modify our data.

	Accuracy		Macro Average F1 Score	
	Validation Dataset	Test Dataset	Validation Dataset	Test Dataset
Week 9 Model	57%	52%	0.46	0.39
Week 10 Model	70%	52%	0.70	0.46

With all three techniques implemented, the validation accuracy increased to a promising 70%. This is much higher than the 57% validation accuracy achieved with the previous model. The macro average for

Group Forecast and Furious
Casey Barrasso, Joshua Previte, Geovanni Jones

the Week 10 model is also much higher than the Week 9 model. This is likely due to the increase of sample size (SMOTE and bootstrapping both helped with this) compared to the previous training dataset, which was severely lacking observation count. The conversion of historical price lags to quantitative variables also provided the model with additional information, which helped inform more accurate predictions. Due to increases in both performance metrics on the validation set, the Week 10 model will be deployed.

When evaluated on the test dataset, accuracy was the same as the previous week's model, both at 52%. This suggests the model is still heavily overfitting on the training and validation datasets, and is unable to extend similar performance to the test dataset.

Our secondary performance metric, the macro average F1 score, did increase from 0.39 to 0.46 in week 10 after the implemented data techniques. The performance across the two classes was much more evenly distributed, with the lowest class f1 score increasing from 0.11 to 0.27. Interestingly, the model implemented this week did a much better job implementing the "decrease" class over the "increase" class, a change from previous weeks' models. This suggests the SMOTE technique did indeed work, but it now caused the model to be slightly too cautious in predicting increases in stock price direction.